QSM603FCP Ubuntu 编译&烧录指导

智能产品

版本：1.0.0

日期：2026-10-26

状态：临时文件

上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司
上海市松江区泗泾镇外婆泾路 8 号　邮编：201601
电话：+86 21 5108 6236　　邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：http://www.quectel.com/cn/support/sales.htm。

如需技术支持或反馈我司技术文档中的问题，请随时登录网址：
http://www.quectel.com/cn/support/technical.htm 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您的。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他硬软件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 © 上海移远通信技术股份有限公司 2026，保留一切权利。
Copyright © Quectel Wireless Solutions Co., Ltd. 2026.

文档历史

修订记录

版本	日期	作者	变更描述
-	2026-10-26	Andrew Xu	文档创建
1.0.0	2026-10-26	Andrew Xu	临时版本

目录

文档历史 3
目录 4
表格索引 5
图片索引 6
1 引言 7
2 开发环境 8
2.1. 编译环境 8
2.2. 工具安装和启动 9
2.2.1. Windows 环境工具准备 9
2.2.2. Ubuntu 环境工具准备 9
2.2.2.1. 使用 Docker 搭建 Ubuntu 24.04 编译环境 9
2.2.2.2. 将 Ubuntu 24.04 镜像导入 Docker 10
3 编译与烧录 12
3.1. 访问 GitLab 克隆代码 12
3.2. 编译和烧录流程 12
3.2.1. 常用编译命令 12
3.2.2. 烧录固件包 13
3.2.2.1. 在 Windows 上烧录固件包 13
4 开发示例 18
4.1. SDK 编译 18
5 分区介绍 20
6 附录 参考文档及术语缩写 21

表格索引

表 1：编译环境 8
表 2：参考文档 21
表 3：术语缩写 21

图片索引

图 1：USB 驱动文件 9
图 2：Docker 版本查询示例 10
图 3：Ubuntu 编译环境 Docker 镜像 10
图 4：烧录目录文件 14
图 5：启动 Genio 烧录工具并等待设备 14
图 6：进入下载模式及开始擦除分区 15
图 7：擦除分区及写入 LUN2 镜像 15
图 8：继续写入镜像 16
图 9：烧录完成 17

1 引言

本文档主要介绍移远通信QSM603FCP设备的编译和烧录方法，包括开发环境、编译烧录流程、开发示例以及分区介绍。

备注

1. 本文档适用于运行 Ubuntu 操作系统的 QSM603FCP 设备。
2. 项目开发注意事项说明，可从 FTP（ftp://202.111.194.162，路径：*/Common*）获取《智能模块客户软件开发注意事项》进行参考。
3. FTP（ftp://202.111.194.162）用于存放工具、驱动和镜像等相关资料供用户下载使用，无其他用途。
4. 由于 PDF 格式特性，文档中部分代码块可能因页面宽度限制自动换行。直接从 PDF 复制代码到编辑器时，可能引入非预期的换行符（\n），导致代码无法正常运行。

   建议操作：

   若需从 PDF 复制，粘贴后请注意完成如下动作：
   - 手动删除复制粘贴后代码中的多余换行符。
   - 或通过代码编辑器的替换功能批量删除多余换行符（注意保留语义换行）。

2 开发环境

2.1. 编译环境

表 1：编译环境

描述	版本	目的
为确保系统构建效率，Linux 开发工作站推荐硬件配置如下： **CPU**：Intel i7-2600 @ 3.4 GHz 或以上 **内存**：16 GB RAM 或以上 **硬盘**：固态硬盘（SSD），预留 400 GB 或以上可用空间 **Linux 发行版**：Ubuntu 24.04 LTS	Ubuntu 24.04 LTS (64-bit)	Linux 主机端系统构建
Windows 系统	Windows 10 或更高版本	运行固件刷写工具

备注

1. 推荐使用基于 Ubuntu 24.04 LTS（64-bit）标准环境的 Docker 镜像进行源码编译。移远通信官方已针对该版本完成完整验证。若使用其他历史版本（如 14.04/16.04/19.04/18.04 LTS），移远通信对此暂未做完整验证。
2. 编译前需要获取相关代码，请联系移远通信技术支持获取 GitLab 访问权限来下载 SDK。

2.2. 工具安装和启动

2.2.1. Windows 环境工具准备

1. Python

   需要使用 Python 3.12.x 通过 Windows PC 的 Genio Tools 工具烧录 QSM603FCP 固件。

   - 从 https://www.python.org/downloads/下载最新的 Python 3.12.x 版本并安装；
   - 安装成功后，将 Python 的安装路径添加到 Windows 的 PATH 环境变量中。

2. USB 驱动

   从 FTP（ftp://202.111.194.162，路径：*Linux/Tools*）下载 USB 驱动并安装，驱动文件示例如下：

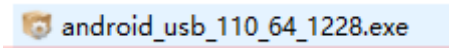


图 1：USB 驱动文件

3. 烧录工具 -- Genio Tools

```
$ py -3.12 -m pip uninstall genio-tools
$ py -3.12 -m pip install genio-tools==1.7.0
```

2.2.2. Ubuntu 环境工具准备

2.2.2.1. 使用 Docker 搭建 Ubuntu 24.04 编译环境

2.2.2.1.1. 安装 Docker

1. 执行以下命令更新 APT 软件包索引：

```
$ sudo apt-get update
```

2. 执行以下命令在安装 Docker：

```
$ sudo apt install docker.io qemu-user-static binfmt-support
```

3. 执行以下命令验证 Docker 安装状态：

```
$ sudo docker -v
```

若 Docker 安装成功，终端将返回当前安装的版本信息，示例如下（版本号以实际查询结果为准）：

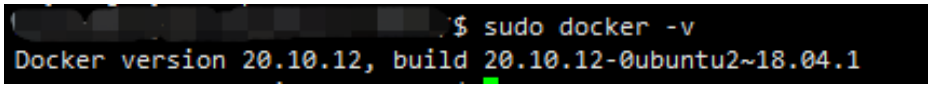


图 2：Docker 版本查询示例

2.2.2.2. 将 Ubuntu 24.04 镜像导入 Docker

请按照如下步骤使用移远通信提供的 Docker 镜像来搭建 Ubuntu 24.04 编译环境。

1. 从 FTP（ftp://202.111.194.162/，路径：*Linux/Tools*）上下载 Docker 镜像压缩包 *smartlinux-qsm603fcp-ubuntu-24.04-v1.0.tar.gz*。

2. 执行以下命令将镜像导入至 Docker：

```
$ sudo docker load -i smartlinux-qsm603fcp-ubuntu-24.04-v1.0.tar.gz
```

3. 执行以下命令查看导入到 Docker 的镜像：

```
$ sudo docker images
```

导入的 Docker 镜像如下图所示。

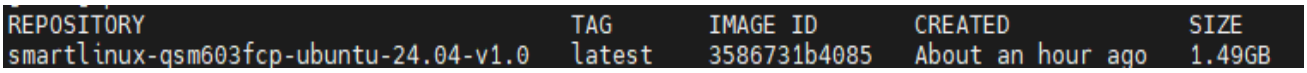


图 3：Ubuntu 编译环境 Docker 镜像

4. 首先按照***第 3.1 章***获取代码，然后在宿主机代码根目录执行以下命令运行镜像（**$PWD** 表示当前代码目录）：

```
$ sudo docker run --privileged -it \
    -e QCS_IN_DOCKER=1 \
    -v "$PWD:/home/user/mt8371_ubuntu_bu01_v25.1.1" \
    -w /home/user/mt8371_ubuntu_bu01_v25.1.1 \
    smartlinux-qsm603fcp-ubuntu-24.04-v1.0:latest /bin/bash
```

备注

Docker 常用命令与注意事项：

1. 导入的 Docker 镜像已默认创建了“user”普通用户，若无特殊需求，则无需再创建用户。
2. Docker 容器中有 **sudo** 命令，若普通用户需要使用 root 权限，使用 **sudo -i** 进入 root 用户，使用 **exit** 命令退回到 user 用户。
3. 通过 **sudo -i** 切换到 root 用户后，执行 **exit** 返回 user 用户；在 user 用户下执行 **exit** 退出 Docker 容器。
4. 在宿主机上执行以下命令可查看目前已经创建 Docker 容器的 ID：

   ```
   sudo docker ps -a
   ```

   对应的 Docker 容器 ID 示例如下（后续命令中的 *CONTAINER_ID* 应替换为实际值）：

   ```
   CONTAINER ID   IMAGE          COMMAND                  CREATED              STATUS                          PORTS     NAMES
   2a0e5e9ae91f   d0cb46621816   "/bin/bash"              About a minute ago   Exited (0) 51 seconds ago                 sleepy_margulis
   ```

5. 在宿主机上执行以下命令启用 Docker 容器：

   ```
   sudo docker start CONTAINER_ID
   ```

6. 在宿主机上执行以下命令进入 Docker 容器：

   ```
   sudo docker attach CONTAINER_ID
   ```

7. 在宿主机上执行以下命令停用 Docker 容器：

   ```
   sudo docker stop CONTAINER_ID
   ```

8. 请优先使用移远提供的 Docker 镜像进行代码编译，因为编译环境所需的工具已预装于镜像中。

3 编译与烧录

3.1. 访问 GitLab 克隆代码

1. 请联系移远通信技术支持获取 GitLab 账号；
2. 登录 GitLab（网址：https://git-master.quectel.com/）；
3. 将 SSH 密钥添加至 GitLab 服务器；
4. 在命令行键入以下内容：

```
git clone \
ssh://git@git-master.quectel.com:8407/smart_new/mt8371_ubuntu_bu01_v25.1.1/mt8371_ubuntu_bu01_v25.1.1.git
```

详细操作可参考*文档 [1]*。

3.2. 编译和烧录流程

3.2.1. 常用编译命令

1. 执行如下命令将分支切换到 *QSM603FCP_rl*：

```
$ git branch -a
QSM603FCP_rl
$ git checkout QSM603FCP_rl
```

2. 执行如下命令进行整编：

```
$ cd /home/user/mt8371_ubuntu_bu01_v25.1.1
$ source build.sh
$ build all
```

编译完成后，生成的镜像位于 *bin/mtk-mt8371/*（此目录即为烧录的整包）。

3. 执行如下命令清除所有内容：

```
$ cd /home/user/mt8371_ubuntu_bu01_v25.1.1
$ source build.sh
$ buildclean
```

4. 执行如下命令编译其他组件或软件包：

```
build kernel
build bootloader
build package <包名>
build rootfs
build pack
```

组件编译产物位于 *build_dir/mtk-mt8371/*目录下。

- 修改后应执行相应组件编译，再执行 **build rootfs**、**build pack** 生成烧录包；
- U-Boot 修改后还需执行 **build platform** 更新 FIP。

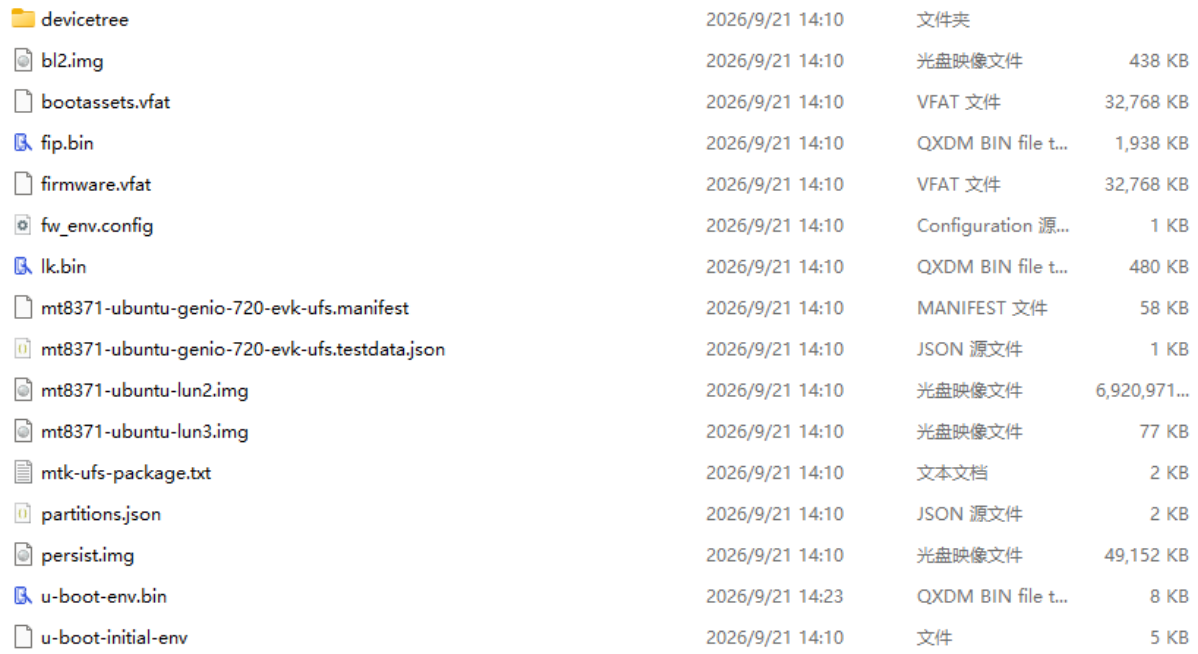
备注

无论代码位于本地 Ubuntu 编译机还是 Docker 容器中，都不能使用 root 权限来操作代码。

3.2.2. 烧录固件包

3.2.2.1. 在 Windows 上烧录固件包

编译完成后，固件包位于*bin/mtk-mt8371*目录下，主要内容如下：



devicetree	2026/9/21 14:10	文件夹	
bl2.img	2026/9/21 14:10	光盘映像文件	438 KB
bootassets.vfat	2026/9/21 14:10	VFAT 文件	32,768 KB
fip.bin	2026/9/21 14:10	QXDM BIN file t...	1,938 KB
firmware.vfat	2026/9/21 14:10	VFAT 文件	32,768 KB
fw_env.config	2026/9/21 14:10	Configuration 源...	1 KB
lk.bin	2026/9/21 14:10	QXDM BIN file t...	480 KB
mt8371-ubuntu-genio-720-evk-ufs.manifest	2026/9/21 14:10	MANIFEST 文件	58 KB
mt8371-ubuntu-genio-720-evk-ufs.testdata.json	2026/9/21 14:10	JSON 源文件	1 KB
mt8371-ubuntu-lun2.img	2026/9/21 14:10	光盘映像文件	6,920,971...
mt8371-ubuntu-lun3.img	2026/9/21 14:10	光盘映像文件	77 KB
mtk-ufs-package.txt	2026/9/21 14:10	文本文档	2 KB
partitions.json	2026/9/21 14:10	JSON 源文件	2 KB
persist.img	2026/9/21 14:10	光盘映像文件	49,152 KB
u-boot-env.bin	2026/9/21 14:23	QXDM BIN file t...	8 KB
u-boot-initial-env	2026/9/21 14:10	文件	5 KB

图 4：烧录目录文件

1. 在 Windows 命令行进入完整烧录包目录并执行以下命令：

```
cd mtk-mt8371
genio-flash
```

工具等待设备的界面如下图所示：



```
PS Y:\smartlinux_sg560d\rl\MT8371_Ubuntu_BU01_V25.1.1\bin\mtk-mt8371> genio-flash
Genio Tools: v1.7.0
Yocto Image:
       name:     MediaTek MT8371 Canonical Ubuntu 24.04 Desktop (mt8371-ubuntu)
       distro:   Ubuntu 24.04 (ubuntu)
       codename: noble
       machine:  genio-720-evk-ufs
       overlays: ['video.dtbo', 'apusys.dtbo', 'camera-gc13a0-s5k3p9sx04-auto-dual-std.dtbo', 'display-edp-fhd.dtbo', 'display-lvds.dtbo', 'ufs.dtbo']

WARNING:aiot:Cannot find any FTDI device
WARNING:aiot:Unable to find and reset the board. Possible causes are:
1. This is not a Genio 350/700 EVK, nor a Pumpkin board.
2. The board port UART0 is not connected.
3. The UART0 port is being opened by another tool, such as TeraTerm on Windows.
You can now manually reset the board into DOWNLOAD mode.

INFO:aiot:Continue flashing...
Looking for MediaTek SoC matching USB device 0e8d:0003
```

图 5：启动 Genio 烧录工具并等待设备

2. 进入下载模式：将工控板的 BOOT 引脚短接，接上 USB，模块会进入下载模式；看到开始擦除分区（*图 6*）后接上电源线。

```
Opening COM10 using baudrate=115200
Connected to MediaTek SoC: hw_code[0x8189]
Sending bootstrap to address: 0x2001000
Jumping to bootstrap at address 0x2001000 in AArch64 mode
erasing mmc0
< waiting for any device >
Erasing 'mmc0'                                  (bootloader) request sz: 0xedc800000, real erase len: 0x0
```

图 6：进入下载模式及开始擦除分区

3. 后续烧录流程如*图 7*、*图 8* 和*图 9* 所示。

```
PS Y:\smartlinux_sg560d\rl\MT8371_Ubuntu_BU01_V25.1.1\bin\mtk-mt8371> genio-flash
Genio Tools: v1.7.0
Yocto Image:
       name:     MediaTek MT8371 Canonical Ubuntu 24.04 Desktop (mt8371-ubuntu)
       distro:   Ubuntu 24.04 (ubuntu)
       codename: noble
       machine:  genio-720-evk-ufs
       overlays: ['ufs.dtbo', 'display-lvds.dtbo', 'camera-gc13a0-s5k3p9sx04-auto-dual-std.dtbo', 'video.dtbo', 'display-edp-fhd.dtbo', 'apusys.dtbo']

WARNING:aiot:Cannot find any FTDI device
WARNING:aiot:Unable to find and reset the board. Possible causes are:
1. This is not a Genio 350/700 EVK, nor a Pumpkin board.
2. The board port UART0 is not connected.
3. The UART0 port is being opened by another tool, such as TeraTerm on Windows.
You can now manually reset the board into DOWNLOAD mode.

INFO:aiot:Continue flashing...
Looking for MediaTek SoC matching USB device 0e8d:0003
Opening COM10 using baudrate=115200
Connected to MediaTek SoC: hw_code[0x8189]
Sending bootstrap to address: 0x2001000
Jumping to bootstrap at address 0x2001000 in AArch64 mode
erasing mmc0
< waiting for any device >
Erasing 'mmc0'                                  (bootloader) request sz: 0xedc800000, real erase len: 0x0
OKAY [  0.384s]
Finished. Total time: 0.388s
erasing mmc1
Erasing 'mmc1'                                  (bootloader) request sz: 0x9000000, real erase len: 0x9000000
OKAY [  0.005s]
Finished. Total time: 0.009s
erasing mmc0boot0
Erasing 'mmc0boot0'                             (bootloader) request sz: 0x400000, real erase len: 0x400000
OKAY [  2.063s]
Finished. Total time: 2.067s
erasing mmc0boot1
Erasing 'mmc0boot1'                             (bootloader) request sz: 0x400000, real erase len: 0x400000
OKAY [  2.076s]
Finished. Total time: 2.080s
flashing mmc0=mt8371-ubuntu-lun2.img
Sending sparse 'mmc0' 1/28 (257947 KB)          OKAY [  8.185s]
Writing 'mmc0'                                  OKAY [  2.417s]
Sending sparse 'mmc0' 2/28 (262140 KB)          OKAY [  8.597s]
Writing 'mmc0'                                  OKAY [  2.046s]
Sending sparse 'mmc0' 3/28 (260656 KB)          OKAY [  8.547s]
Writing 'mmc0'                                  OKAY [  2.080s]
Sending sparse 'mmc0' 4/28 (230244 KB)          OKAY [  7.643s]
Writing 'mmc0'                                  OKAY [  1.801s]
```

图 7：擦除分区及写入 LUN2 镜像

```
Sending sparse 'mmc0' 5/28 (262140 KB)             OKAY [  8.667s]
Writing 'mmc0'                                     OKAY [  2.027s]
Sending sparse 'mmc0' 6/28 (262140 KB)             OKAY [  8.572s]
Writing 'mmc0'                                     OKAY [  1.992s]
Sending sparse 'mmc0' 7/28 (262140 KB)             OKAY [  8.368s]
Writing 'mmc0'                                     OKAY [  1.997s]
Sending sparse 'mmc0' 8/28 (253064 KB)             OKAY [  8.271s]
Writing 'mmc0'                                     OKAY [  1.972s]
Sending sparse 'mmc0' 9/28 (238873 KB)             OKAY [  7.824s]
Writing 'mmc0'                                     OKAY [  1.867s]
Sending sparse 'mmc0' 10/28 (255872 KB)            OKAY [  8.423s]
Writing 'mmc0'                                     OKAY [  1.955s]
Sending sparse 'mmc0' 11/28 (234360 KB)            OKAY [  7.769s]
Writing 'mmc0'                                     OKAY [  1.786s]
Sending sparse 'mmc0' 12/28 (262140 KB)            OKAY [  8.614s]
Writing 'mmc0'                                     OKAY [  1.990s]
Sending sparse 'mmc0' 13/28 (262140 KB)            OKAY [  8.584s]
Writing 'mmc0'                                     OKAY [  1.993s]
Sending sparse 'mmc0' 14/28 (260212 KB)            OKAY [  8.529s]
Writing 'mmc0'                                     OKAY [  1.989s]
Sending sparse 'mmc0' 15/28 (262140 KB)            OKAY [  8.288s]
Writing 'mmc0'                                     OKAY [  1.982s]
Sending sparse 'mmc0' 16/28 (242617 KB)            OKAY [  7.835s]
Writing 'mmc0'                                     OKAY [  1.997s]
Sending sparse 'mmc0' 17/28 (239448 KB)            OKAY [  7.618s]
Writing 'mmc0'                                     OKAY [  1.877s]
Sending sparse 'mmc0' 18/28 (262140 KB)            OKAY [  8.396s]
Writing 'mmc0'                                     OKAY [  2.004s]
Sending sparse 'mmc0' 19/28 (262140 KB)            OKAY [  8.382s]
Writing 'mmc0'                                     OKAY [  1.992s]
Sending sparse 'mmc0' 20/28 (262140 KB)            OKAY [  8.368s]
Writing 'mmc0'                                     OKAY [  1.991s]
Sending sparse 'mmc0' 21/28 (262140 KB)            OKAY [  8.326s]
Writing 'mmc0'                                     OKAY [  1.984s]
Sending sparse 'mmc0' 22/28 (262140 KB)            OKAY [  8.375s]
Writing 'mmc0'                                     OKAY [  1.993s]
Sending sparse 'mmc0' 23/28 (262140 KB)            OKAY [  8.396s]
Writing 'mmc0'                                     OKAY [  1.997s]
Sending sparse 'mmc0' 24/28 (262143 KB)            OKAY [  8.834s]
Writing 'mmc0'                                     OKAY [  2.683s]
Sending sparse 'mmc0' 25/28 (249168 KB)            OKAY [  8.140s]
Writing 'mmc0'                                     OKAY [  2.047s]
Sending sparse 'mmc0' 26/28 (256734 KB)            OKAY [  8.263s]
Writing 'mmc0'                                     OKAY [  2.033s]
Sending sparse 'mmc0' 27/28 (259061 KB)            OKAY [  8.416s]
Writing 'mmc0'                                     OKAY [  2.359s]
Sending sparse 'mmc0' 28/28 (12744 KB)             OKAY [  0.417s]
Writing 'mmc0'                                     OKAY [  0.292s]
Finished. Total time: 281.991s
flashing mmc1=mt8371-ubuntu-lun3.img
Warning: skip copying mmc1 image avb footer due to sparse image.
```

图 8：继续写入镜像

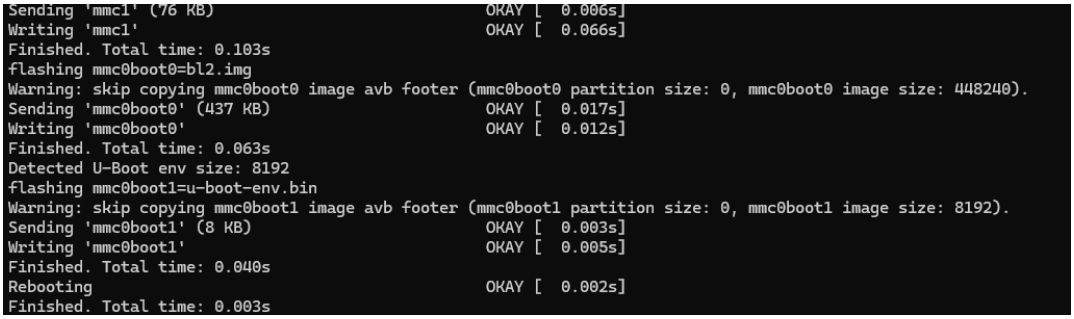

Sending 'mmc1' (76 KB) OKAY [0.006s]
Writing 'mmc1' OKAY [0.066s]
Finished. Total time: 0.103s
flashing mmc0boot0=bl2.img
Warning: skip copying mmc0boot0 image avb footer (mmc0boot0 partition size: 0, mmc0boot0 image size: 448240).
Sending 'mmc0boot0' (437 KB) OKAY [0.017s]
Writing 'mmc0boot0' OKAY [0.012s]
Finished. Total time: 0.063s
Detected U-Boot env size: 8192
flashing mmc0boot1=u-boot-env.bin
Warning: skip copying mmc0boot1 image avb footer (mmc0boot1 partition size: 0, mmc0boot1 image size: 8192).
Sending 'mmc0boot1' (8 KB) OKAY [0.003s]
Writing 'mmc0boot1' OKAY [0.005s]
Finished. Total time: 0.040s
Rebooting OKAY [0.002s]
Finished. Total time: 0.003s

图 9：烧录完成

4 开发示例

4.1. SDK 编译

本章节以 helloworld（Makefile）和 apptest（Autoconf）为例。当前 Ubuntu 基线为示例代码，需先按下述路径添加源码及适配当前编译框架的板级包 Makefile，再执行示例命令。

在编译容器的代码根目录执行：

```
$ source build.sh
```

1. 通过 Makefile 编译 *helloworld*

   - 源码：*target/mtk/mt8371/package/application/helloworld/src/*（包含应用自身的 Makefile）
   - 包 Makefile：*target/mtk/mt8371/package/application/helloworld/Makefile*

   添加板级包后，执行如下命令编译*helloworld*：

   ```
   $ build package helloworld
   ```

   编译工作目录为 *build_dir/mtk-mt8371/packages/helloworld/*。配置安装到*/usr/bin* 后，程序暂存到 *build_dir/mtk-mt8371/package-staging/usr/bin/helloworld* 目录。

2. 通过 Autoconf 编译 *apptest*

   - 源码：*target/mtk/mt8371/package/application/apptest/src/*
   - 包 Makefile：*target/mtk/mt8371/package/application/apptest/Makefile*

   添加板级包后，执行如下命令编译 *apptest*：

   ```
   $ build package apptest
   ```

   编译工作目录为 *build_dir/mtk-mt8371/packages/apptest/*。配置安装到*/usr/bin* 后，程序暂存到 *build_dir/mtk-mt8371/package-staging/usr/bin/apptest* 目录。

3. 默认安装 *helloworld* 和 *apptest* 到系统

   在两个板级包 Makefile 各自的 *Package/helloworld* 和 *Package/apptest* 定义中配置如下选项即可安装到系统：

   ```
   DEFAULT:=y
   ROOTFS_INSTALL:=y
   ```

   此时，执行如下命令进行整编：

   ```
   $ build all
   ```

备注

有关本基线编译命令的详细信息，请执行 **build help**。

5 分区介绍

分区模板 *partitions.json* 和 LUN 布局配置位于如下目录：

- *mt8371_ubuntu_bu01_v25.1.1/target/mtk/mt8371/image/partitions.json*
- *mt8371_ubuntu_bu01_v25.1.1/target/mtk/mt8371/image/luns/*

不允许随意修改默认分区，如需修改，请联系移远通信技术支持。

6 附录 参考文档及术语缩写

表 2：参考文档

文档名称
[1] Quectel_智能模块_GitLab_用户指导

表 3：术语缩写

缩写	英文全称	中文全称
APT	Advanced Package Tool	高级包管理工具
CPU	Central Processing Unit	中央处理器
FIP	Firmware Image Package	固件镜像包
FTP	File Transfer Protocol	文件传输协议
ID	Identifier	标识符
LTS	Long-Term Support	长期支持
LUN	Logical Unit	逻辑单元
PC	Personal Computer	个人电脑
SDK	Software Development Kit	软件开发工具包
SSD	Solid-State Disk	固态硬盘
SSH	Secure Shell	安全外壳协议
USB	Universal Serial Bus	通用串行总线